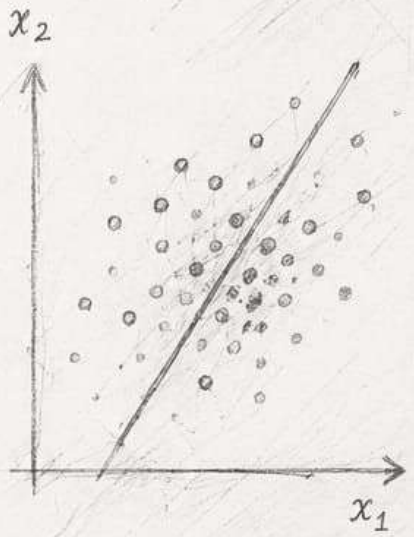
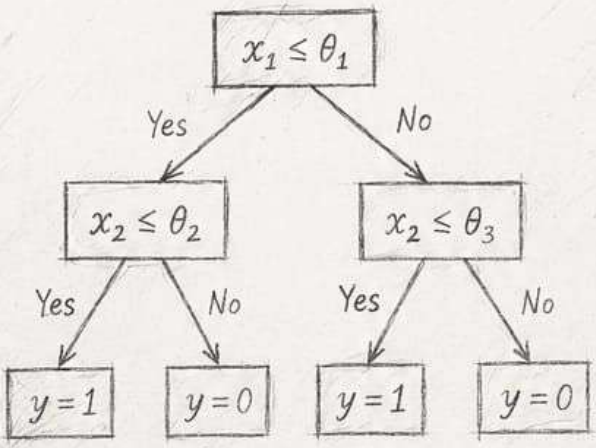
MACHINE LEARNING

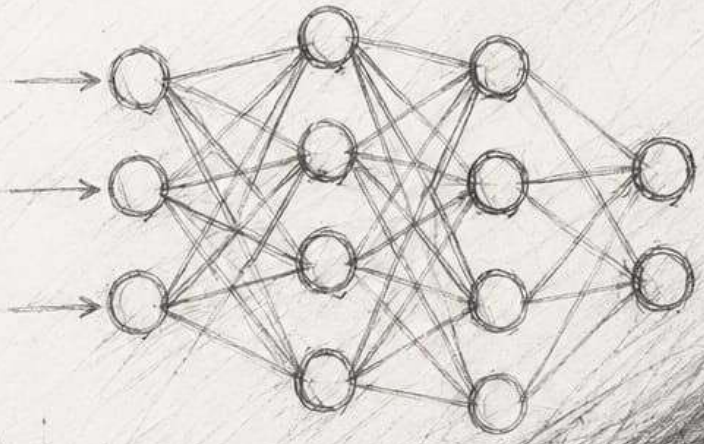
A PRACTICAL APPROACH

Concepts, Algorithms, and Real-World Applications



$$f(x) = \sigma(w^T x + b)$$





YOUR NAME

Introduction To Machine Learning Algorithms

What is Machine Learning?

- Machine Learning is a branch of AI that allows computers to learn from data and make predictions or decisions without being explicitly programmed.

ML = Learning from data + Improving with experience

ML Workflow

Data Collection → Data Preprocessing → Model Training → Evaluation → Prediction / Deployment

Types of Machine Learning

1. Supervised Learning : Uses labeled data to predict output.
   → Regression , Classification
2. Unsupervised Learning : Uses unlabeled data to find patterns.
   → Clustering , Association
3. Reinforcement Learning : Agent learns by interacting with environment and getting rewards or penalties.

Applications

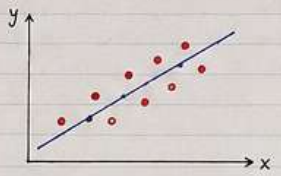
- Email Spam Detection
- House Price Prediction
- Recommendation Systems
- Fraud Detection
- Self Driving Cars
- Healthcare Diagnosis

Supervised Learning Algorithms

1) Linear Regression

- Used for continuous output
- Finds best fit line

Equation : $y = mx + c$

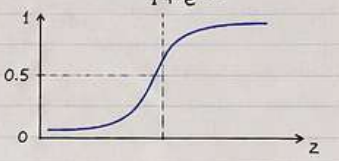


2) Logistic Regression

- Used for classification
- Output is probability (0 to 1)

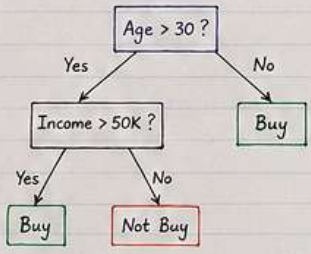
Sigmoid Function :

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



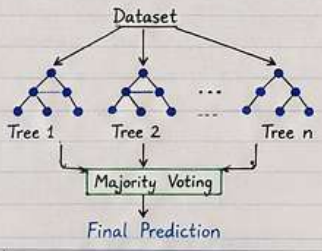
3) Decision Tree

- Tree like structure
- Easy to understand



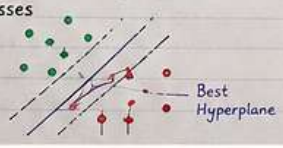
4) Random Forest

- Collection of multiple decision trees
- More accurate and reduces overfitting



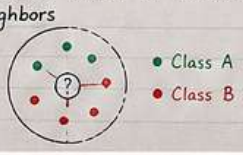
5) Support Vector Machine (SVM)

- Finds best boundary (hyperplane) between classes



6) K-Nearest Neighbors (KNN)

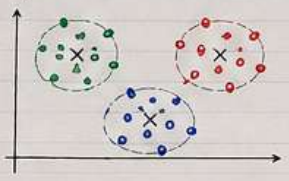
- Classifies based on nearest neighbors



Unsupervised Learning Algorithms

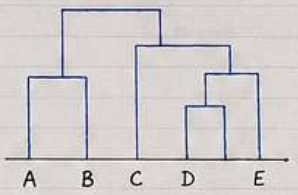
1) K-Means Clustering

- Groups data into K clusters
- Minimizes distance between points and cluster center



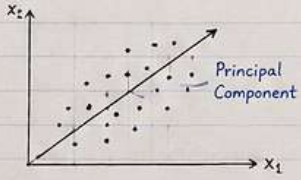
2) Hierarchical Clustering

- Creates a tree (dendrogram) of clusters



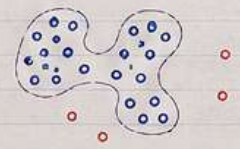
3) Principal Component Analysis (PCA)

- Reduces dimensionality
- Keeps most important features



4) DBSCAN

- Density based clustering
- Finds clusters of arbitrary shape



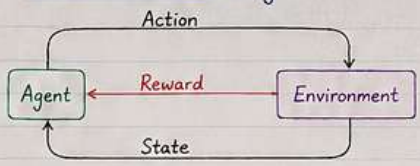
Key Concepts

- Overfitting : Model learns noise
- Underfitting : Model too simple
- Bias : Error due to wrong assumptions
- Variance : Error due to sensitivity
- Train / Test Split : Evaluate model

- Accuracy = $\frac{TP + TN}{(TP + TN + FP + FN}$
- Precision = $\frac{TP}{(TP + FP)}$
- Recall = $\frac{TP}{(TP + FN}$
- F1-Score = $\frac{2 \times (\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})}$

TP : True Positive	FP : False Positive
TN : True Negative	FN : False Negative

Reinforcement Learning



- Agent learns by trial and error
- Goal is to maximize cumulative reward

How to Choose an Algorithm?

Problem Type	Examples	Algorithms
Regression	House Price Prediction	Linear Regression, Ridge, Lasso
Classification	Spam Detection Disease Prediction	Logistic Regression, SVM, Random Forest, KNN, Naive Bayes
Clustering	Customer Segmentation	K-Means, Hierarchical, DBSCAN
Dimensionality Reduction	Data Visualization	PCA, t-SNE
Reinforcement Learning	Game Playing Robotics	Q-Learning, SARSA, Deep Q-Network

Final Notes

- ☑ More data usually → better performance
- ☑ Good quality data > complex model
- ☑ Feature Engineering is very important
- ☑ Always validate your model

Practice + Consistency = Mastery

Machine Learning Algorithms Notes

1 Definition:

A brief explanation of the algorithm's purpose.

2 Mathematical Equation:

Formula of the algorithm -

Geometrical Interpretation:

Visual representation of the algorithm

Geometrical Interpretation:

Visual representation of the algorithm

4 When to Use:

Situations where the algorithm is appropriate.

5 Code Implementation:

Example code snippet (Python, R)

When to Use:

Situations where the algorithm is appropriate.

7 Assumptions:

Key assumptions of the algorithm

8 Advantages:

Benefits of using the algorithm

8 Disadvantages:

Limitations and downsides

9 Hyperparametors:

Key hyperparameters of the algorithm

10 Real-world Applications:

Practical use-cases

Datasets You'll Use

• (Classification)

Age	Gender	Salary	Status
50	Male	5000 usd	1 (happy)
30	Male	100 usd	0 (not)

• (NLP)

Text / comment / review	Label
The product was good	Positive
It was bad, not as per requirements	negative

• (Regression)

H/year	Rooms	Area	Price
10	10	3.4	4000 usd
2	20	10.20	10000 usd

• (Clustering)

Brand	Ram	Batter	Clusters
Oppo	4 gb	5000	
Apple	10 gb	1000	

Classification Algorithms

"For classification problems, we'll cover these algorithms:"

1. Logistic Regression
2. K-Nearest Neighbors (KNN)
3. Decision Tree Classifier
4. Random Forest Classifier
5. Support Vector Machines (SVM)
6. Naive Bayes
7. Gradient Boosting (XGBoost, LightGBM) ⚡

Use Cases: Spam Detection, Disease Prediction, Customer Churn.

Regression Algorithms

"For predicting continuous values, we'll explore regression algorithms:"

1.Linear Regression

2.Polynomial Regression

3.Ridge & Lasso Regression

4.Decision Tree Regressor

5.Random Forest Regressor

6.Support Vector Regression (SVR)

Use Cases: House Price Prediction, Stock Market Forecasting.

LINEAR REGRESSION

1. DEFINITION

Linear Regression is a supervised learning algorithm used to model the relationship between a dependent variable (target) and one or more independent variables (features) using a straight line.

2. MATHEMATICAL EQUATION

For one variable (simple linear regression):

$$y = mx + b$$

For multiple variables (multiple linear regression):

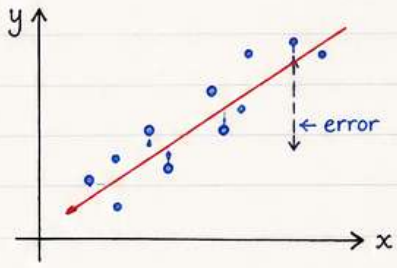
$$y = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

Where,

- y = predicted output
- x_i = input features
- w_i = weights (coefficients)
- b = bias (intercept)

3. GEOMETRICAL INTERPRETATION

- In 2D: It is a best-fit straight line.
- In 3D: It becomes a plane.
- In higher dimensions: It becomes a hyperplane.



The algorithm tries to minimize the vertical distance between data points and the line (error).

4. WHEN TO USE

- When output is continuous (numeric).
- When relationship between variables is approximately linear.
- For forecasting and trend analysis.

Examples:

- House price prediction
- Salary prediction
- Sales forecasting

5. CODE IMPLEMENTATION (PYTHON – SCIKIT-LEARN)

```python
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
# import libraries

# Sample data
X = [[1], [2], [3], [4], [5]]
y = [2, 4, 6, 8, 10]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
# training and testing split

# Model
model = LinearRegression()
model.fit(X_train, y_train)
# create model and train

# Prediction
y_pred = model.predict(X_test)
# predict on test data

# Evaluation
print("MSE:", mean_squared_error(y_test, y_pred))
print("Slope:", model.coef_)
print("Intercept:", model.intercept_)
# evaluate the model
```

6. ASSUMPTIONS

1. Linear relationship between X and y
2. No multicollinearity (in multiple regression)
3. Homoscedasticity (constant variance of errors)
4. Independence of observations
5. Normally distributed residuals

7. ADVANTAGES

- Simple and easy to implement
- Fast training
- Interpretable results
- Works well on linearly separable data

8. DISADVANTAGES

- Poor performance on non-linear data
- Sensitive to outliers
- Assumes linear relationship (often unrealistic)
- Can underfit complex datasets

9. HYPERPARAMETERS

(for sklearn implementation)

- fit_intercept → whether to calculate intercept
- copy_X → copy or overwrite data
- n_jobs → parallel computation (if applicable)

10. REAL-WORLD APPLICATIONS


Stock price trend prediction


Real estate price estimation


Salary prediction based on experience


Demand forecasting in business


Fuel consumption estimation


Agriculture yield prediction

★ LOGISTIC REGRESSION ★

1. DEFINITION

Logistic Regression is a supervised machine learning algorithm used for classification problems.

It predicts the probability that an input belongs to a particular class.

It uses the Sigmoid Function to output values between 0 and 1.

2. MATHEMATICAL EQUATION

- Linear Combination

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

- Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

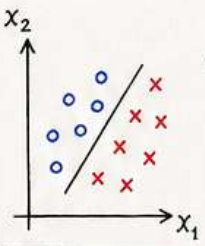
Where:
- z = linear combination
- $\sigma(z)$ = probability
- w = weights
- b = bias / intercept

- Final Prediction

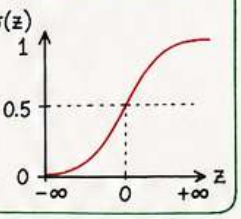
$$\hat{y} = \begin{cases} 1 & \text{if } \sigma(z) \geq 0.5 \\ 0 & \text{if } \sigma(z) < 0.5 \end{cases}$$

3. GEOMETRICAL INTERPRETATION

- Logistic Regression creates a decision boundary.
- In 2D → a straight line separating classes.
- In 3D → a plane.
- Uses sigmoid curve to map outputs into probabilities.



Sigmoid Curve



4. WHEN TO USE

Use Logistic Regression when:
- Output is categorical (binary)
- Binary classification problems
- Need probability scores
- Data is linearly separable

Examples:
- Email spam detection
- Disease prediction
- Customer churn prediction
- Fraud detection

5. CODE IMPLEMENTATION (PYTHON - SCIKIT-LEARN)

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score                    # Import libraries

# Sample Data
X = [[1], [2], [3], [4], [5], [6]]
y = [0, 0, 0, 1, 1, 1]                                        # Sample data

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
)                                                             # Split data into training & testing set

# Model
model = LogisticRegression()                                  # Create model

# Train
model.fit(X_train, y_train)                                   # Train the model

# Prediction
y_pred = model.predict(X_test)                                # Predict on test data

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))            # Model accuracy

# Probability
print(model.predict_proba([[3.5]]))                           # Probability of class [0, 1] for input 3.5
```

6. ASSUMPTIONS

- Binary outcome variable
- Independent observations
- No multicollinearity
- Linear relationship between log-odds and features
- Large sample size preferred

7. ADVANTAGES

- Simple and fast
- Probability output
- Easy to interpret
- Works well for binary classification
- Less prone to overfitting

8. DISADVANTAGES

- Cannot solve complex non-linear problems
- Sensitive to outliers
- Requires feature engineering sometimes
- Assumes linear decision boundary

9. HYPERPARAMETERS

Important hyperparameters in sklearn:
- penalty → regularization type (l_1, l_2)
- C → regularization strength
- solver → optimization algorithm
- max_iter → maximum iterations
- class_weight → handle imbalanced data

10. REAL-WORLD APPLICATIONS

- Spam Email Detection
- Disease Diagnosis
- Credit Card Fraud Detection
- Customer Churn Prediction
- Product Recommendation
- Cybersecurity Threat Detection

DECISION TREE

1. DEFINITION

A Decision Tree is a supervised machine learning algorithm used for both classification and regression tasks.

It works like a tree structure where decisions are made based on conditions.

- Internal Nodes → feature tests
- Branches → outcomes of tests
- Leaf Nodes → final prediction/output

It mimics human decision-making.

2. MATHEMATICAL EQUATION

- Entropy Formula

$$\text{Entropy}(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

- Information Gain

$$IG(S,A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

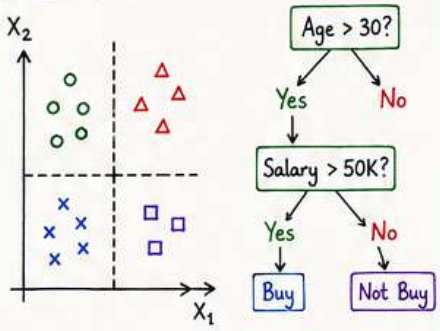
Where,

- p_i = probability of class i
- Entropy measures impurity
- Information Gain measures reduction in impurity

3. GEOMETRICAL INTERPRETATION

- Decision Trees divide feature space into rectangular regions.
- Each split creates a decision boundary.
- Final regions correspond to predicted classes/values.

Example :



4. WHEN TO USE

Use Decision Trees when:

- Need interpretable models
- Data contains non-linear relationships
- Both numerical and categorical data exist
- Feature importance is required

Examples :

- Loan approval systems
- Medical diagnosis
- Customer segmentation
- Fraud detection

5. CODE IMPLEMENTATION (PYTHON - SCIKIT-LEARN)

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Sample Dataset
X = [[25, 40000],
     [35, 60000],
     [45, 80000],
     [20, 20000]]
y = [0, 1, 1, 0]

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Model
model = DecisionTreeClassifier()
# Train
model.fit(X_train, y_train)
# Prediction
y_pred = model.predict(X_test)
# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

6. ASSUMPTIONS

- No assumptions about data distribution
- Works with non-linear relationships
- Features should meaningfully split data
- Large trees may overfit

7. ADVANTAGES

- ✓ Easy to understand and visualize
- ✓ Handles non-linear data
- ✓ Works with categorical & numerical features
- ✓ No feature scaling required
- ✓ Feature importance available

8. DISADVANTAGES

- x Overfitting problem
- x Unstable with small data changes
- x Can become very complex
- x Biased toward dominant classes

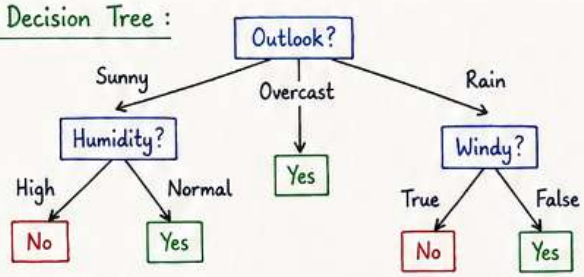
9. HYPERPARAMETERS

Hyperparameter	Description
criterion	gini or entropy (function to split)
max_depth	maximum depth of the tree
min_samples_split	minimum samples required to split an internal node
min_samples_leaf	minimum samples required at a leaf node
max_features	number of features to consider when looking for best split

10. REAL-WORLD APPLICATIONS

- Loan Approval Prediction
- Disease Diagnosis
- Fraud Detection
- Customer Segmentation
- Agriculture Decision Systems
- Recommendation Systems

Example of Decision Tree :



Internal Node
Branch
Leaf Node (Yes)
Leaf Node (No)

Decision Trees are simple, powerful and interpretable but must be pruned or limited to avoid overfitting!

✳ K-Nearest Neighbors (KNN) ✳

K-Nearest Neighbors is a supervised machine learning algorithm used for:

- Classification
- Regression

It predicts the output based on the nearest data points in the dataset.

1. Definition

KNN is a non-parametric and instance-based learning algorithm that classifies a new data point based on the majority class among its nearest neighbors.

Key Concepts:

- K → number of nearest neighbors
- Distance Metric → measures similarity
- Majority Voting → for classification
- Averaging → for regression

KNN stores all training data and makes predictions during testing.

2. Mathematical Equation

- Euclidean Distance Formula

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

- Manhattan Distance Formula

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

- Classification Rule

Predict the class with the highest frequency among K nearest neighbors.

Where,

x_i = feature value of point x
y_i = feature value of point y
n = number of features

3. Geometrical Interpretation

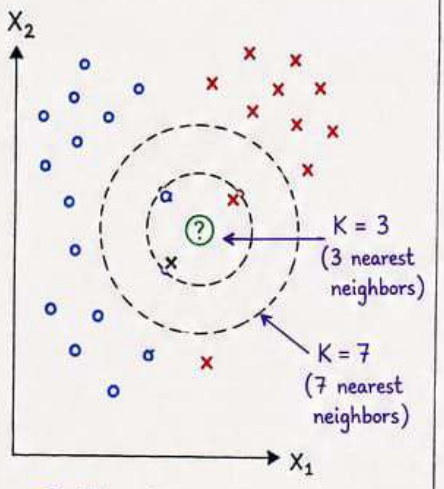
- KNN finds the nearest points in feature space
- Distance determines similarity
- Nearby points likely belong to the same class
- Decision boundary depends on K value

Types of Distance Metrics:

- Euclidean Distance
- Manhattan Distance
- Minkowski Distance
- Hamming Distance

Behavior:

- Small K → sensitive to noise
- Large K → smoother boundary



4. When To Use

Use KNN when:

- Dataset is small to medium-sized
- Data has clear patterns
- Non-linear decision boundaries exist
- Fast training is needed
- Recommendation systems are used

Examples:

- Recommendation systems
- Pattern recognition
- Image classification
- Medical diagnosis

5. Code Implementation

(Python - Scikit-Learn)

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Sample Dataset
X = [[2, 3],
     [3, 4],
     [1, 1],
     [8, 9],
     [9, 10],
     [10, 8]]
y = [0, 0, 0, 1, 1, 1]

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
# Model
model = KNeighborsClassifier(n_neighbors=3)
# Train
model.fit(X_train, y_train)
# Prediction
y_pred = model.predict(X_test)
# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

6. Assumptions

- Similar data points exist near each other
- Data is properly scaled
- Noise is limited
- Meaningful distance metric is chosen

7. Advantages

- ✓ Simple and easy to understand
- ✓ No training phase required
- ✓ Handles non-linear data
- ✓ Works for multi-class problems
- ✓ Flexible distance metrics

8. Disadvantages

- × Slow prediction time
- × Memory intensive
- × Sensitive to irrelevant features
- × Sensitive to scaling
- × Performance decreases with large datasets

9. Hyperparameters

- n_neighbors → number of neighbors
- weights → uniform or distance
- metric → distance metric
- p → power parameter for Minkowski distance
- algorithm → auto, ball_tree, kd_tree, brute

10. Real-World Applications

Recommendation Systems | Image Recognition | Medical Diagnosis | Customer Segmentation | Handwriting Detection | Pattern Recognition

RANDOM FOREST

1. DEFINITION

Random Forest is an ensemble machine learning algorithm that combines multiple decision trees to improve prediction accuracy and reduce overfitting.

- Classification → Majority Voting
- Regression → Average Prediction

It uses:

- Bagging (Bootstrap Sampling)
- Random Feature Selection

2. MATHEMATICAL EQUATION

- Classification (Voting)

$$\hat{y} = \text{mode}(T_1(x), T_2(x), \ldots, T_n(x))$$

- Regression (Averaging)

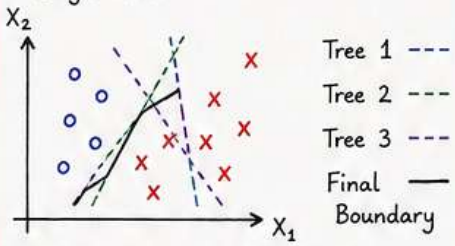
$$\hat{y} = \frac{1}{n} \sum_{i=1}^{n} T_i(x)$$

Where,

- $T_i(x)$ = prediction from i^{th} tree
- n = total number of trees
- mode() = majority voting

3. GEOMETRICAL INTERPRETATION

- Multiple decision trees create multiple decision boundaries.
- Final prediction is based on combined boundaries.
- Produces smoother and more generalized regions than a single tree.



Each tree learns differently from random subsets of data/features. Ensemble combines all tree decisions.

4. WHEN TO USE

Use Random Forest when:

- Need high accuracy
- Dataset is large
- Data contains non-linear relationships
- Overfitting occurs in Decision Trees
- Feature importance is needed

Examples:

- Fraud detection
- Disease prediction
- Recommendation systems
- Stock market prediction

5. CODE IMPLEMENTATION (PYTHON – SCIKIT-LEARN)

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Sample Dataset
X = [[25, 40000],
     [35, 60000],
     [45, 80000],
     [20, 20000],
     [50, 90000]]
y = [0, 1, 1, 0, 1]

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size = 0.2, random_state = 42)

# Model
model = RandomForestClassifier(n_estimators = 100,
                               random_state = 42)

# Train
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

6. ASSUMPTIONS

- No strict assumptions about data distribution
- Trees should be diverse
- Enough trees improve stability
- Independent observations preferred

7. ADVANTAGES

- ✓ High accuracy
- ✓ Reduces overfitting
- ✓ Handles non-linear data
- ✓ Works with large datasets
- ✓ Feature importance available
- ✓ Handles missing values better

8. DISADVANTAGES

- X Slower than Decision Trees
- X More memory consumption
- X Less interpretable
- X Large models can become computationally expensive

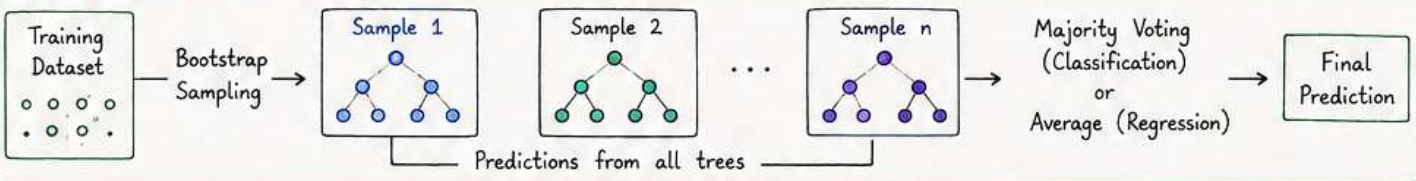
9. HYPERPARAMETERS

Hyperparameter	Description
n_estimators	Number of trees in the forest
max_depth	Maximum depth of the trees
min_samples_split	Minimum samples required to split an internal node
min_samples_leaf	Minimum samples required at a leaf node
max_features	Number of features considered at each split
bootstrap	Whether bootstrap sampling (with replacement) is used

10. REAL-WORLD APPLICATIONS

- Credit Card Fraud Detection
- Disease Prediction
- Stock Market Analysis
- Recommendation Systems
- Agriculture Yield Prediction
- Autonomous Vehicle Systems

HOW RANDOM FOREST WORKS?



* Naive Bayes *

Naive Bayes is a supervised machine learning algorithm based on Bayes' Theorem used for:

- Classification
- Probabilistic Prediction

It predicts the class of data using probability and prior knowledge.

1. Definition

Naive Bayes is a probabilistic classification algorithm that assumes all input features are independent of each other.

Key Concepts:

- Bayes Theorem
- Prior Probability
- Posterior Probability
- Likelihood
- Feature Independence Assumption

It is called "Naive" because it assumes features are independent.

2. Mathematical Equation

Bayes Theorem

$$P(A|B) = \frac{P(B|A)\,P(A)}{P(B)}$$

Naive Bayes Formula

$$P(C|X) = \frac{P(X|C)\,P(C)}{P(X)}$$

Where:

$P(C\|X)$	=	Posterior Probability
$P(X\|C)$	=	Likelihood
$P(C)$	=	Prior Probability
$P(X)$	=	Evidence
C	=	Class
X	=	Feature vector

3. Geometrical Interpretation

- Naive Bayes calculates probabilities for each class
- Assigns the class with the highest probability
- Works well in high-dimensional datasets
- Assumes features contribute independently

Types of Naive Bayes:

- Gaussian Naive Bayes
- Multinomial Naive Bayes
- Bernoulli Naive Bayes

Behavior:

- Fast probabilistic classification
- Effective for text data

4. When to Use

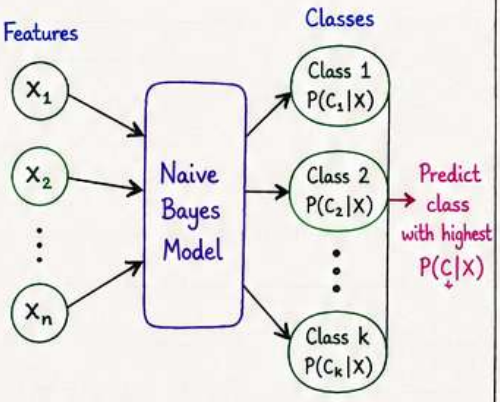
Use Naive Bayes when:

- Dataset is high-dimensional
- Text classification is required
- Fast prediction is needed
- Features are mostly independent
- Limited training data exists

Examples:

- Spam filtering
- Sentiment analysis
- Document classification
- Recommendation systems

5. Geometrical Interpretation (Diagram)



The class having the highest posterior probability is selected.

6. Code Implementation

(Python - Scikit-Learn)

```python
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Sample Dataset
X = [2, 3],
    [3, 4],
    [1, 1],
    [8, 9],
    [9, 10],        y = [0, 0, 0, 1, 1, 1]
    [10, 8]]

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size = 0.2, random_state = 42)
# Model
model = GaussianNB()
# Train
model.fit(X_train, y_train)
# Prediction
y_pred = model.predict(X_test)
# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

7. Assumptions

- Features are independent
- All relevant features are included
- No strong correlation between features
- Data distribution matches model assumptions

8. Advantages

- ✓ Simple and fast
- ✓ Works well with high-dimensional data
- ✓ Performs well on small datasets
- ✓ Excellent for text classification
- ✓ Requires less training data

9. Disadvantages

- × Strong independence assumption
- × Sensitive to correlated features
- × Probability estimates may be inaccurate
- × Zero-frequency problem can occur
- × Less effective for complex relationships

10. Hyperparameters

- var_smoothing → stability for GaussianNB
- alpha → smoothing parameter
- fit_prior → whether to learn class priors
- class_prior → prior probabilities

11. Real-World Applications

Spam Email Detection · Sentiment Analysis · News Classification · Recommendation Systems · Medical Diagnosis · Fraud Detection

✳ Support Vector Machine (SVM) ✳

Support Vector Machine is a supervised machine learning algorithm used for:

- Classification
- Regression
- Outlier Detection

It finds the optimal boundary (hyperplane) that best separates data points into classes.

1. Definition

Support Vector Machine tries to find the best decision boundary with the maximum margin between classes.

Key Concepts:

- Hyperplane → decision boundary
- Support Vectors → nearest data points to boundary
- Margin → distance between boundary and support vectors

SVM focuses on maximizing the margin for better generalization.

2. Mathematical Equation

- Hyperplane Equation

$$w^T x + b = 0$$

- Classification Rule

$$f(x) = \text{sign}(w^T x + b)$$

- Margin Formula

$$\text{Margin} = \frac{2}{\|w\|}$$

Where,

- w = weight vector
- b = bias
- x = input feature vector
- sign() → predicts class label

3. Geometrical Interpretation

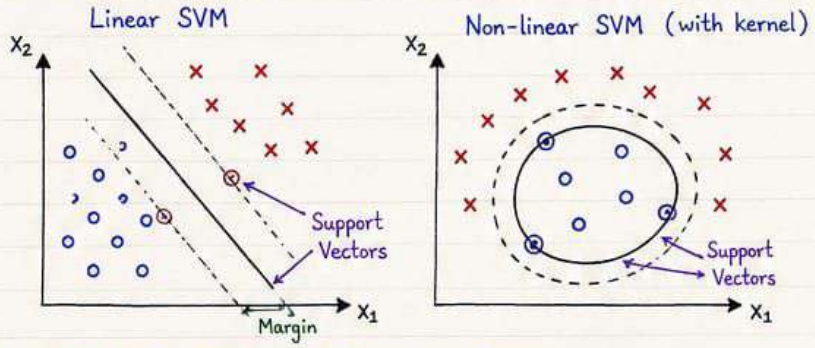
- SVM creates a separating hyperplane
- Goal → maximize distance between classes
- Support vectors define the boundary
- Kernel trick allows non-linear separation

Types:

- Linear SVM → straight boundary
- Non-linear SVM → curved boundary using kernels

Common Kernels:

- Linear
- Polynomial
- RBF (Gaussian)
- Sigmoid



4. When to Use

Use SVM when:

- Dataset is small to medium-sized
- High-dimensional data exists
- Clear margin of separation exists
- Text classification problems
- Complex classification boundaries

Examples:

- Face recognition
- Spam detection
- Image classification
- Bioinformatics

5. Code Implementation (Python - Scikit-learn)

```python
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Sample Dataset
X = [[2, 3],
     [3, 4],
     [1, 1],
     [8, 9],
     [9, 10],
     [10, 8]]
y = [0, 0, 0, 1, 1, 1]

# Split Dataset
X_train, X_test, y_train, y test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Model
model = SVC(kernel='linear')
# Train
model.fit(X_train, y_train)
# Prediction
y_pred = model.predict(X_test)
# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

6. Assumptions

- Data is separable
- Independent observations
- Feature scaling improves performance
- Works better with clear margins between classes

7. Advantages

- ✓ Effective in high-dimensional spaces
- ✓ Works well with small datasets
- ✓ Memory efficient
- ✓ Handles non-linear data using kernels
- ✓ Strong generalization capability

8. Disadvantages

- × Slow for very large datasets
- × Sensitive to noise/outliers
- × Difficult to interpret
- × Choosing kernel can be complex
- × Computationally expensive

9. Hyperparameters

Important sklearn hyperparameters:

- kernel → linear, poly, rbf, sigmoid
- C → regularization parameter
- gamma → kernel coefficient
- degree → polynomial degree
- probability → probability estimates

10. Real-World Applications

- Spam Email Detection
- Face Recognition
- Cancer Classification
- Image Classification
- Gene Expression Analysis
- Cybersecurity Intrusion Detection

* Gradient Boosting *

1. Definition

Gradient Boosting is an ensemble learning algorithm that builds models sequentially. Each new model tries to correct the errors made by the previous models.

Key Concepts:

- Weak learners (usually decision trees)
- Sequential learning
- Minimizes loss function
- Gradient Descent Optimization

2. Mathematical Equation

Gradient Boosting model:

$$F_M(x) = F_0(x) + \sum_{m=1}^{M} \gamma_m h_m(x)$$

Where,

- $F_0(x)$ = Initial model (constant)
- $h_m(x)$ = m^{th} weak learner
- γ_m = Learning rate
- M = Number of boosting stages

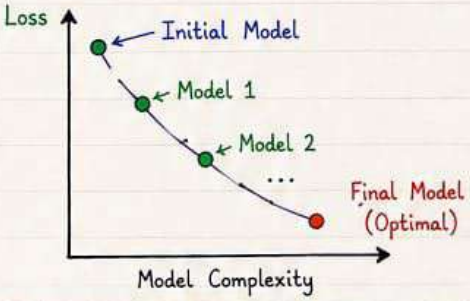
At each step, a new model $h_m(x)$ is trained to predict the negative gradient of the loss function.

Objective (Loss Minimization):

$$\text{Minimize } L(y, F_M(x))$$

3. Geometrical Interpretation

- Gradient Boosting moves in the direction of steepest descent to minimize the loss.
- Each new model corrects the residual errors of the previous model.
- The final model is a strong learner built by additive combination of weak learners.



4. When to Use

- When high predictive accuracy is needed
- When data has complex patterns
- For structured/tabular data
- When handling missing values
- When overfitting can be controlled using regularization

Examples:

- Click-through rate prediction
- Fraud detection
- Customer churn prediction
- Demand forecasting

5. Code Implementation (Python - Scikit-learn)

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score

# Sample Dataset
X = [[2, 3],
     [3, 4],
     [1, 1],
     [8, 9],
     [9, 10],
     [10, 8]]
y = [0, 0, 0, 1, 1, 1]

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size = 0.2, random_state = 42)

# Model
model = GradientBoostingClassifier(n_estimators = 100,
    learning_rate = 0.1, max_depth = 3, random_state = 42)

# Train
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

6. Assumptions

- The relationship between features and target can be learned iteratively.
- Errors of previous models can be corrected by new models.
- Data is representative of the problem.
- Requires enough data for learning meaningful patterns.

7. Advantages

- ✓ High predictive performance
- ✓ Handles complex relationships
- ✓ Works with different loss functions
- ✓ Less prone to overfitting than single models (with tuning)
- ✓ Handles missing values
- ✓ Feature importance available

8. Disadvantages

- ✗ Sensitive to hyperparameters
- ✗ Training time can be high
- ✗ Can overfit if not tuned
- ✗ Not ideal for very large datasets
- ✗ Harder to interpret than simple models

9. Hyperparameters

- `n_estimators` → Number of boosting stages
- `learning_rate` → Shrinks contribution of each tree
- `max_depth` → Maximum depth of trees
- `subsample` → Fraction of samples used for training each tree
- `loss` → Loss function (e.g., "log_loss", "squared_error")
- `min_samples_split` → Minimum samples required to split an internal node

10. Real-World Applications

Email Spam Detection


E-commerce Sales Prediction


Credit Score Assessment


Healthcare Diagnosis


Stock Price Prediction


Fraud Detection

✳ Machine Learning Important Formulas Cheat Sheet ✳

1. Linear Regression

- Prediction Equation

$$y = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$$

- Cost Function (MSE)

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- Gradient Descent Update

$$w := w - \alpha\frac{\partial J}{\partial w}$$

2. Logistic Regression

- Sigmoid Function

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

- Binary Cross Entropy Loss

$$L = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i\log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right]$$

3. Decision Tree

- Entropy

$$\text{Entropy}(S) = -\sum_{i=1}^{c} p_i\log_2(p_i)$$

- Information Gain

$$\text{IG}(S,A) = \text{Entropy}(S) - \sum_{v}\frac{|S_v|}{|S|}\text{Entropy}(S_v)$$

- Gini Index

$$\text{Gini} = 1 - \sum_{i=1}^{c} p_i^2$$

4. Random Forest

- Classification Voting

$$\hat{y} = \text{mode}(T_1(x), T_2(x), \ldots, T_n(x))$$

- Regression Averaging

$$\hat{y} = \frac{1}{n}\sum_{i=1}^{n} T_i(x)$$

5. Support Vector Machine (SVM)

- Hyperplane

$$w^Tx + b = 0$$

- Margin

$$\text{Margin} = \frac{2}{|w|}$$

- Classification Function

$$f(x) = \text{sign}(w^Tx + b)$$

6. Naive Bayes

- Bayes Theorem

$$P(A|B) = \frac{P(B|A)\,P(A)}{P(B)}$$

- Naive Bayes Formula

$$P(C|X) \propto P(C)\prod_{i=1}^{n} P(x_i|C)$$

7. K-Nearest Neighbors (KNN)

- Euclidean Distance

$$d(x,y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

- Manhattan Distance

$$d(x,y) = \sum_{i=1}^{n}|x_i - y_i|$$

8. K-Means Clustering

- Centroid Update

$$\mu_k = \frac{1}{|C_k|}\sum_{x_i \in C_k} x_i$$

- Objective Function

$$J = \sum_{k=1}^{K}\sum_{x_i \in C_k}|x_i - \mu_k|^2$$

9. Principal Component Analysis (PCA)

- Covariance Matrix

$$\text{Cov}(X) = \frac{1}{n}(X - \bar{X})^T(X - \bar{X})$$

- Eigenvalue Equation

$$Av = \lambda v$$

10. Neural Networks

- Neuron Output

$$z = \sum_{i=1}^{n} w_ix_i + b$$

- Activation Function (ReLU)

$$f(x) = \max(0, x)$$

- Softmax

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

11. Deep Learning

- Backpropagation

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y}\cdot\frac{\partial y}{\partial w}$$

- Adam Optimizer

$$\theta_t = \theta_{t-1} - \alpha\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

12. Evaluation Metrics

- Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

- Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

- F1 Score

$$F1 = 2\cdot\frac{\text{Precision}\cdot\text{Recall}}{\text{Precision} + \text{Recall}}$$

13. Probability & Statistics

- Mean

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

- Variance

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2$$

- Standard Deviation

$$\sigma = \sqrt{\sigma^2}$$

14. Regularization

- L1 Regularization (Lasso)

$$L = \text{Loss} + \lambda\sum|w_i|$$

- L2 Regularization (Ridge)

$$L = \text{Loss} + \lambda\sum w_i^2$$

15. Reinforcement Learning

- Q-Learning Update

$$Q(s,a) = Q(s,a) + \alpha\left[r + \gamma\max_{a'}Q(s',a') - Q(s,a)\right]$$

16. Transformer / Attention

- Attention Formula

$$\text{Attention}(Q,K,V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Positional Encoding

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

17. GAN (Generative Adversarial Network)

- GAN Objective

$$\min_G\max_D V(D,G) = E_{x\sim P_{data}}[\log D(x)] + E_{z\sim P_z}[\log(1 - D(G(z)))]$$

18. Autoencoder

- Reconstruction Loss

$$L(x,\hat{x}) = |x - \bar{x}|^2$$

19. Gradient Descent Variants

- Batch Gradient Descent

$$\theta = \theta - \alpha\nabla J(\theta)$$

- Momentum

$$v_t = \beta v_{t-1} + (1-\beta)\nabla J(\theta)$$

20. Cosine Similarity

- Cosine Similarity

$$\text{Cosine}(x,y) = \frac{x\cdot y}{|x|\,|y|}$$

✳ Note: x = input features, y = true value, $\hat{y}$ = predicted value, w = weights, b = bias, α = learning rate, n = number of samples, c = number of classes. ✳

IMPORTANT MATHEMATICAL DEFINITIONS IN DATA SCIENCE

1. Gradient Descent

$$\theta = \bar{\theta} - \alpha \nabla J(\theta)$$

2. Normal Distribution

$$X \sim \mathcal{N}(\mu, \sigma^2)$$

3. Z-Score

$$Z = \frac{X - \mu}{\sigma}$$

4. Sigmoid Function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

5. Correlation

$$r = \frac{\text{cov}(X, Y)}{\sigma_X \sigma_Y}$$

6. Cosine Similarity

$$\cos\theta = \frac{A \cdot B}{\|A\| \, \|B\|}$$

7. Naive Bayes

$$P(C|X) = \frac{P(X|C)\,P(C)}{P(X)}$$

8. Maximum Likelihood Estimation

$$L(\theta) = \prod_{i=1}^{\infty} P(x_i|\theta)$$

9. Ordinary Least Squares (OLS)

$$\hat{y} = \beta_0 + \beta_1 X$$

10. F1 Score

$$F1 = \frac{2PR}{P + R}$$

11. ReLU Activation

$$f(x) = \max(0, x)$$

12. Eigenvectors

$$Ax = \lambda x$$

13. R^2 Score

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

14. Softmax Function

$$\sigma_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

15. Mean Squared Error (MSE)

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

16. Entropy

$$H = -\sum_{i=1}^{n} p_i \log p_i$$

17. KL Divergence

$$D_{KL}(P \| Q) = \sum_{i=1}^{n} P(x_i) \log \frac{P(x_i)}{Q(x_i)}$$

18. Log Loss

$$L = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]$$

19. Singular Value Decomposition (SVD)

$$A = U \Sigma V^T$$

20. Lagrange Multiplier

$$L = f(x, y) + \lambda g(x, y)$$

21. Support Vector Machine (SVM)

$$w \cdot x + b = 0$$

22. Linear Regression

$$\hat{y} = \beta_0 + \beta_1 X$$

23. Multiple Linear Regression

$$\hat{y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n$$

24. Principal Component Analysis (PCA)

$$Z = XW$$

25. Backpropagation

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial w}$$

26. Adam Optimizer

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

27. Batch Gradient Descent

$$\theta = \theta - \alpha \nabla J(\theta)$$

28. Momentum

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla J(\theta)$$

29. K-Nearest Neighbors (KNN)

- Euclidean: $d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$
- Manhattan: $d(x, y) = \sum_{k=1} |x_i - y_i|$

30. K-Means Clustering

- Centroid Update: $\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$
- Objective: $J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$

31. Evaluation Metrics

- Accuracy $= \frac{TP + TN}{TP + TN + FP + FN}$
- Precision $= \frac{TP}{TP + FP}$
- Recall $= \frac{TP}{TP + FN}$
- F1 Score $= 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$

32. Probability & Statistics

- Mean: $\mu = \frac{1}{n} \sum_{i=1}^{n} x_i$
- Variance: $\sigma^2 = \frac{1}{n} \sum_{i=1}^{n} (x_i - \mu)^2$
- Std. Deviation: $\sigma = \sqrt{\sigma^2}$

33. Regularization

- L1 Regularization (Lasso):
  $L = \text{Loss} + \lambda \sum |w_i|$
- L2 Regularization (Ridge):
  $L = \text{Loss} + \lambda \sum w_i^2$

34. Reinforcement Learning

Q-Learning Update:

$$Q(s,a) = Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$$

35. Transformer / Attention

- Attention: $\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$
- Positional Encoding: $PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_2}}\right)$

36. GAN (Generative Adversarial Network)

GAN Objective:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}}\left[\log D(x)\right] + \mathbb{E}_{z \sim p_z}\left[\log(1 - D(G(z))\right]$$

37. Autoencoder

Reconstruction Loss:

$$L(x, \hat{x}) = \|x - \hat{x}\|^2$$

38. Gradient Descent Variants

- Batch GD: $\theta = \theta - \alpha \nabla J(\theta)$
- Momentum: $v_t = \beta v_{t-1} + (1 - \beta) \nabla J(\theta)$

39. Cosine Similarity

$$\text{Cosine}(x, y) = \frac{x \cdot y}{\|x\| \, \|y\|}$$